프로토콜 버전 협상 과정 분석

개요

이 문서는 Rust libp2p에서 multistream-select 프로토콜을 통한 프로토콜 버전 협상 과정을 상세히 분석합니다. multistream-select는 두 피어 간에 사용할 애플리케이션 계층 프로토콜을 협상하는 데 사용되는 핵심 메커니즘입니다.

multistream-select 프로토콜 개요

기본 개념

multistream-select는 libp2p에서 연결이나 서브스트림에서 사용할 애플리케이션 계층 프로토콜을 협상하는 프로토콜입니다. 이 프로토콜은 두 가지 역할을 정의합니다:

- **Dialer (개시자)**: 프로토콜을 제안하고 협상을 주도하는 능동적 역할
- **Listener (응답자)**: 제안된 프로토콜에 응답하는 수동적 역할

지원되는 버전

```rust
#[derive(Clone, Copy, Debug, PartialEq, Eq, Default)]
pub  enum Version {
/// Version 1 of the multistream-select protocol
#[default]
V1,
/// A "lazy" variant of version 1 that performs 0-RTT negotiation
```

```
V1Lazy,

}
```

Version::V1

- 표준 multistream-select 1.0.0 프로토콜
- 항상 최소 한 번의 전용 라운드트립 메시지 교환 필요
- 프로토콜 협상이 완료된 후 애플리케이션 데이터 전송 가능

Version::V1Lazy

- V1과 동일한 와이어 프로토콜이지만 최적화된 변형
- 0-RTT 협상 지원 (단일 프로토콜만 지원하는 경우)
- Dialer가 프로토콜 협상 데이터를 지연시켜 초기 애플리케이션 데이터와 결합
- Listener는 V1과 동일하게 동작하여 상호 운용성 보장

프로토콜 메시지 구조

메시지 타입

```rust
#[derive(Debug, Clone, PartialEq, Eq)]
pub(crate) enum Message {
/// multistream-select 프로토콜 식별 헤더
Header(HeaderLine),
/// 프로토콜 요청 또는 확인
Protocol(Protocol),
/// 지원되는 프로토콜 목록 요청
```

```
ListProtocols,

/// 지원되는 프로토콜 목록 응답

Protocols(Vec<Protocol>),

/// 요청된 프로토콜을 사용할 수 없음을 알리는 메시지

NotAvailable,

}
```

헤더 라인

```
#[derive(Copy, Clone, Debug, PartialEq, Eq)]

pub(crate) enum HeaderLine {

/// The `/multistream/1.0.0` header line

V1,

}
```

프로토콜 상수

```
/// multistream-select 1.0.0 헤더 메시지의 인코딩된 형태

const MSG_MULTISTREAM_1_0: &[u8] = b"/multistream/1.0.0\n";

/// multistream-select 'na' 메시지의 인코딩된 형태

const MSG_PROTOCOL_NA: &[u8] = b"na\n";

/// multistream-select 'ls' 메시지의 인코딩된 형태
```

```
const MSG_LS: &[u8] = b"ls\n";
```

프로토콜 협상 흐름

Dialer (개시자) 측 협상 과정

1. 헤더 전송 단계

```
State::SendHeader { mut io } => {
let h = HeaderLine::from(*this.version);
Pin::new(&mut io).start_send(Message::Header(h))?;
// 다음 단계로 전환
}
```

2. 프로토콜 제안 단계

```
State::SendProtocol { mut io, protocol } => {
let p = Protocol::try_from(protocol.as_ref())?;
Pin::new(&mut io).start_send(Message::Protocol(p.clone()))?;
tracing::debug!(protocol=%p, "Dialer: Proposed protocol");
}
```

3. 응답 대기 단계

```
State::AwaitProtocol { mut io, protocol } => {
```

```
let msg = Pin::new(&mut io).poll_next(cx)?;

match msg {

Message::Protocol(ref p) if p.as_ref() == protocol.as_ref() => {

// 프로토콜 확인됨 - 협상 성공

let io = Negotiated::completed(io.into_inner());

return Poll::Ready(Ok((protocol, io)));

}

Message::NotAvailable => {

// 프로토콜 거부됨 - 다음 프로토콜 시도

let protocol = this.protocols.next().ok_or(NegotiationError::Failed)?;

*this.state = State::SendProtocol { io, protocol }

}

}

}
```

Listener (응답자) 측 협상 과정

1. 헤더 수신 단계

```
State::RecvHeader { mut io } => {

match io.poll_next_unpin(cx) {

Poll::Ready(Some(Ok(Message::Header(HeaderLine::V1)))) => {

*this.state = State::SendHeader { io }

}
```

```
}

}
```

2. 헤더 응답 단계

```
State::SendHeader { mut io } => {

let msg = Message::Header(HeaderLine::V1);

Pin::new(&mut io).start_send(msg)?;

*this.state = State::Flush { io, protocol: None };

}
```

3. 프로토콜 메시지 수신 단계

```
State::RecvMessage { mut io } => {

match msg {

Message::ListProtocols => {

let supported = this.protocols.iter().map(|(_, p)| p).cloned().collect();

let message = Message::Protocols(supported);

*this.state = State::SendMessage { io, message, protocol: None }

}

Message::Protocol(p) => {

let protocol = this.protocols.iter().find_map(|(name, proto)| {

if &p == proto { Some(name.clone()) } else { None }

});
```

```
let message = if protocol.is_some() {

Message::Protocol(p.clone()) // 확인

} else {

Message::NotAvailable // 거부

};

}

}

}
```

버전별 협상 차이점

V1 표준 협상

1. **Dialer**: 헤더 + 프로토콜 제안 전송
2. **Listener**: 헤더 응답 + 프로토콜 확인/거부
3. **완료**: 양방향 확인 후 애플리케이션 데이터 전송 시작

```
Dialer Listener

| |

|-- /multistream/1.0.0 -->|

|-- /protocol/1.0.0 ----->|

| |

|<-- /multistream/1.0.0 --|

|<-- /protocol/1.0.0 -----|
```

```
| |

|== Application Data ====>|
```

V1Lazy 최적화 협상

단일 프로토콜만 지원하는 경우:

1. **Dialer**: 즉시 프로토콜에 "정착"하고 협상 메시지를 버퍼링
2. **Listener**: V1과 동일하게 동작
3. **0-RTT**: 첫 번째 애플리케이션 데이터와 함께 협상 메시지 전송

```
Version::V1Lazy => {
tracing::debug!(protocol=%p, "Dialer: Expecting proposed protocol");
let hl = HeaderLine::from(Version::V1Lazy);
let io = Negotiated::expecting(io.into_reader(), p, Some(hl));
return Poll::Ready(Ok((protocol, io)));
}
```

프로토콜 식별자 및 버전 관리

프로토콜 식별자 형식

```
impl TryFrom<&str> for Protocol {
fn try_from(value: &str) -> Result<Self, Self::Error> {
if !value.starts_with('/') {
```

```
return Err(ProtocolError::InvalidProtocol);
}
Ok(Protocol(value.to_owned()))
}
}
```

프로토콜 식별자는 반드시 '/'로 시작해야 하며, 일반적인 형식은:

- `/protocol-name/version` (예: `/ipfs/kad/1.0.0`)
- `/libp2p/protocol/version` (예: `/libp2p/autonat/1.0.0`)

실제 사용 예시

```
// Kademlia DHT 프로토콜
pub(crate) const DEFAULT_PROTO_NAME: StreamProtocol =
StreamProtocol::new("/ipfs/kad/1.0.0");

// AutoNAT 프로토콜
pub const DEFAULT_PROTOCOL_NAME: StreamProtocol =
StreamProtocol::new("/libp2p/autonat/1.0.0");
```

협상 완료 및 상태 관리

Negotiated 상태

```
enum State<R> {
```

```
/// 프로토콜 확인을 기다리는 상태 (V1Lazy에서 사용)
Expecting {
io: MessageReader<R>,
header: Option<HeaderLine>,
protocol: Protocol,
},
/// 프로토콜이 합의되어 I/O 통신 가능한 상태
Completed {
io: R,
},
}
```

협상 완료 처리

```
impl<TInner> Negotiated<TInner> {
/// 완료된 상태로 Negotiated 생성
pub(crate) fn completed(io: TInner) -> Self {
Negotiated { state: State::Completed { io } }
}
/// 확인 대기 상태로 Negotiated 생성 (V1Lazy용)
pub(crate) fn expecting(
io: MessageReader<TInner>,
protocol: Protocol,
```

```
header: Option<HeaderLine>,

) -> Self {

Negotiated {

state: State::Expecting { io, protocol, header }

}

}

}
```

오류 처리 및 폴백 메커니즘

협상 오류 타입

```
#[derive(Debug)]

pub enum NegotiationError {

/// 프로토콜 오류 발생

ProtocolError(ProtocolError),

/// 합의할 수 있는 프로토콜이 없어 협상 실패

Failed,

}


#[derive(Debug)]

pub enum ProtocolError {

/// I/O 오류
```

```
IoError(io::Error),

/// 원격에서 잘못된 메시지 수신

InvalidMessage,

/// 프로토콜 이름이 유효하지 않음

InvalidProtocol,

/// 너무 많은 프로토콜 수신

TooManyProtocols,

}
```

폴백 메커니즘

Dialer는 여러 프로토콜을 우선순위 순으로 제안할 수 있습니다:

```
let protocols = vec!["/preferred/2.0.0", "/fallback/1.0.0"];

let (protocol, io) = dialer_select_proto(stream, protocols, Version::V1).await?;
```

1. 첫 번째 프로토콜 제안
2. `NotAvailable` 응답 시 다음 프로토콜 시도
3. 모든 프로토콜이 거부되면 `NegotiationError::Failed` 반환

성능 최적화 고려사항

메시지 인코딩/디코딩

```
impl Message {
```

```rust
/// 메시지를 바이트 표현으로 인코딩
fn encode(&self, dest: &mut BytesMut) {
match self {
Message::Header(HeaderLine::V1) => {
dest.reserve(MSG_MULTISTREAM_1_0.len());
dest.put(MSG_MULTISTREAM_1_0);
}
Message::Protocol(p) => {
let len = p.as_ref().len() + 1; // + 1 for \n
dest.reserve(len);
dest.put(p.0.as_ref());
dest.put_u8(b'\n');
}
// ... 기타 메시지 타입
}
}
}
```

버퍼링 및 배치 처리

- V1Lazy는 협상 메시지와 애플리케이션 데이터를 결합하여 전송
- 길이 구분 메시지 사용으로 효율적인 파싱
- 최대 프로토콜 수 제한 (MAX_PROTOCOLS = 1000)

실제 사용 사례

Swarm에서의 통합

```
// swarm/src/connection.rs에서
let (info, stream) = multistream_select::dialer_select_proto(
substream,
protocols,
self.upgrade_version,
).await.map_err(to_stream_upgrade_error)?;

let (info, stream) = multistream_select::listener_select_proto(
substream,
protocols
).await.map_err(to_stream_upgrade_error)?;
```

프로토콜 핸들러에서의 사용

각 libp2p 프로토콜은 고유한 식별자를 정의하고 multistream-select를 통해 협상됩니다:

- Kademlia: `/ipfs/kad/1.0.0`
- Gossipsub: `/meshsub/1.1.0`, `/meshsub/1.0.0`
- Identify: `/ipfs/id/1.0.0`
- Ping: `/ipfs/ping/1.0.0`

보안 고려사항

프로토콜 검증

```
impl TryFrom<Bytes> for Protocol {
fn try_from(value: Bytes) -> Result<Self, Self::Error> {
if !value.as_ref().starts_with(b"/") {
return Err(ProtocolError::InvalidProtocol);
}
let protocol_as_string = String::from_utf8(value.to_vec())
.map_err(|_| ProtocolError::InvalidProtocol)?;
Ok(Protocol(protocol_as_string))
}
}
```

DoS 방어

- 최대 프로토콜 수 제한으로 메모리 소모 공격 방지
- 잘못된 메시지 형식 검증
- 타임아웃 처리로 무한 대기 방지

결론

multistream-select 프로토콜은 libp2p의 핵심 구성 요소로서 효율적이고 확장 가능한 프로토콜 협상을 제공합니다. V1과 V1Lazy 버전을 통해 표준 호환성과 성능 최적화 사이의 균형을 제공하며, 강력한 오류 처리와 폴백 메커니즘을 통해 안정적인 P2P 통신을 지원합니다.